

MemSeer: Leverage Memory Failure Distinctions and Multi-Grained Prediction in Ultra-Scale Heterogeneous X86/ARM Clusters

Yunfei Gu[†], Yixuan Liu[†], Xinyuan Wu[†], Bo Shao[†], Chentao Wu^{†*}, Shiyi Li[‡], Jieru Zhao[†],
Jie Li[†], Minyi Guo[†], Kunlin Yang[§], Wengui Zhang[§], Feilong Lin[§]

[†]Shanghai Jiao Tong University, Shanghai, China

[‡]Harbin Institute of Technology, Shenzhen, China

[§]Huawei Technologies Co., Ltd., China

*Corresponding Authors: wuct@cs.sjtu.edu.cn

Abstract—In high-performance ultra-scale cloud computing, heterogeneous clusters consisting of x86 and ARM architecture platforms have become increasingly common to boost performance and energy efficiency. Ensuring high availability in these environments is crucial for meeting service-level agreements. However, DRAM failures, a primary cause of server downtimes, present significant challenges to reliability, availability, and serviceability. This paper provides an in-depth analysis of memory failure characteristics across cross-architecture platforms in large-scale heterogeneous clusters. We introduce MemSeer, an AIOps-integrated tool that utilizes a multi-grained memory failure prediction approach for x86/ARM heterogeneous clusters. MemSeer improves the F1-score by 17.3% and increases recall by an average of 27% across different lead times compared to state-of-the-art methods. These advancements show great promise in reducing memory failures in cluster environments, decreasing VM interruptions by up to 42.7% and averaging 24.2% in real-world implementations.

Index Terms—DRAM Reliability, Memory Failures, Prediction, Heterogeneous Clusters

I. INTRODUCTION

In the rapid development of high-performance cloud computing, heterogeneous clusters are increasingly used to optimize diverse computing resources and enhance performance and energy efficiency [1]. High Availability (HA) is crucial, with cloud providers setting availability targets in Service Level Agreements (SLAs) [2]. However, hardware failures, especially in Dynamic Random Access Memory (DRAM), jeopardize Reliability, Availability, and Serviceability (RAS) [3], risking data integrity as memory scales and workloads grow. Our research at Huawei shows that memory failures contribute to 57% of server downtimes, as illustrated in Figure 1. To mitigate DRAM errors, chips use Error Correction Code (ECC) methods like Single Bit Error Correction and Double Bit Error Detection Code (SEC/DED) [4], Single Device Data Correction (SDDC) [5], and Chipkill ECC [6]. When errors exceed ECC capabilities, they are categorized as *Correctable Errors (CEs)* and *Uncorrectable Errors (UEs)*, with UEs often leading to data loss or system crashes.

To enhance resilience and reduce memory failures, various fault-tolerance strategies are incorporated into BIOS firmware and AIOps systems [7–10]. Additionally, machine learning (ML)-based memory failure prediction schemes have emerged as a promising solution, leveraging historical error logs to extract CEs and UEs information and offer predictive failure analyses [11, 12]. This enables proactive maintenance

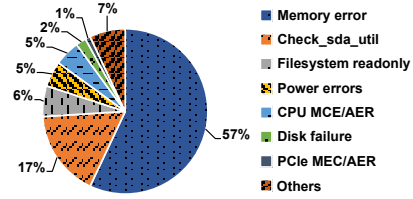


Fig. 1: The breakdown of the causes of server crashes in Huawei clusters.

options such as Dual In-line Memory Module (DIMM) replacement [13, 14], Page Offlining in the operating system by retiring faulty memory regions [15, 16], and migration of Virtual Machines (VMs) across servers before system collapse [17]. Research [14, 18–24] investigates memory faults in cells, rows, and columns to foresee failures. State-of-the-art studies [13–16, 25–27] focus on exploiting more effective indicators for improving memory failure prediction precision and recall. Du *et al.* [14, 25] pioneeringly discuss the bit-level CE patterns and define a new risky CE indicator for the future UEs occurrence prediction. [27] utilizes the concept of CE storms as an important feature to predict the DRAM-caused node unavailability. [26] goes deeper into the bit-level DQ and beats information within the cacheline and presents the hierarchical memory failure prediction (HiMFP) framework and attempts to extend their work across various CPU architectures [28]. However, there are remaining challenges:

- ① Servers with distinct x86 and ARM CPU architectures may exhibit notable differences in DIMM error distribution patterns, affecting predictive performance.
- ② Long-term lead time and fine-grained memory failure predictions perform poorly and offer limited value for actual system fault tolerance.

To address these challenges, we further leverage the spatial and temporal distinctions in memory failure characteristics between x86 and ARM architectures in ultra-scale heterogeneous clusters derived from real-world data centers, including Intel SkyLake, Cascade Lake, and Huawei Kunpeng servers. Subsequently, we introduce MemSeer, a multi-grained memory failure prediction approach that forecasts failures at the DIMM, server, and micro (page and row) levels, showing promise for VM migration and page offlining to prevent system crashes.

Our contributions are summarized as follows:

- We present an in-depth exploration of memory failure logs across ultra-scale heterogeneous clusters, including x86 and ARM architectures, drawing from a dataset comprising about

2 million DIMMs from over 120,000 servers.

- We propose MemSeer and employ corresponding feature engineering to realize the leading multi-grained memory failure prediction, including DIMM-level, server-level and micro-level (i.e., page-level, row-level).
- We conduct comprehensive experiments to evaluate MemSeer’s predictive performance and cost benefits from page offlining and live VM migrations based on multi-grained failure prediction. It reduces VM interruptions by 24.2% on average in real-world implementations.

II. BACKGROUND

A. Memory Organization and Memory Access

The memory system in a server is hierarchical, with DIMMs connected to the CPU through the Integrated Memory Controller (IMC) via channels, as demonstrated in Figure 2. A DIMM consists of multiple DRAM chips organized in ranks for parallelism, with each rank containing 16 data-storing chips and 2 ECC chips. Each chip has multiple banks organized into two-dimensional arrays of cells, indexed by rows and columns, with each cell containing several bits based on the DRAM bit width [21, 25, 26]. In a DDR4 DIMM, data is transmitted through the Data Bus (DQ wires), sending 8 beats of data, including ECC error codes, to the IMC across 18 chips. Each beat encompasses 18 x4 cells (4-bit width, which means the DRAM chip is organized to provide 4 bits of data per access), totaling 72 bits (64 bits for data and 8 for ECC). ECC bits identify error locations in DQs and beats [25, 26]. A typical DDR4 DRAM die (8 Gbit) features banks with 64K rows and 1K columns [21]. Due to Die, Rank, and Channel Interleaving technologies in BIOS [29], a row can accommodate data from 2 to 48 4KB pages by distributing data across multiple channels, ranks, and dies for parallel access.

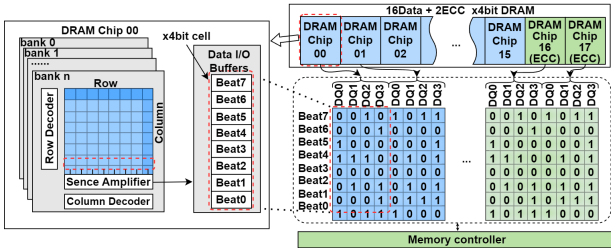


Fig. 2: DIMM components and memory access.

B. Memory Faults, Errors and Failures

Memory Faults refer to the underlying causes of errors, including soft faults and hard faults. (1) **Soft Faults** occur randomly due to strikes by high-energy particles, cosmic rays, etc. However, they may self-recover during subsequent read or write operations. (2) **Hard Faults** refer to irreversible damages in hardware. They develop on the micro-level DRAM circuits, with granularity at the level of rows, columns, banks, or ranks.

Memory Error is a general symptom indicating abnormal behavior in memory, representing instances where one or multiple bits in DRAM read do not match the data that was originally stored. From the perspectives of the Operating System (OS) or AIOps, UEs can be categorized as follows: (1) **Critical UE**: Fatal Errors that lead to service interruptions,

causing OS kernel panic, process interruptions, and server crashes. (2) **Detected and Unconsumed UE**: Errors detected by the IMC during memory access, which have not yet been loaded into the CPU execution path [10, 30]. (3) **Patrol Scrubbing UE**: Errors discovered through the memory patrol scrubbing mechanism, which periodically initiates memory requests to devices at a predefined frequency, triggering a hardware interrupt and notifying the software layer of the corresponding memory address [10, 31].

III. OBSERVATIONS AND MOTIVATION

We analyze memory failure logs from Huawei ultra-scale clusters over three years (2020 Jan. to 2022 Dec.). This dataset includes around 2 million DIMMs from various manufacturers, such as Samsung, Hynix, Micron, Kingston, and Ramaxel. The logs feature timestamps, error types, events, and address details for each CE and UE¹. We extract bit-level information for each CE from the read retry register². A CE storm suppression mechanism is enabled to mitigate node unresponsiveness from excessive CE interruptions and CPU resource exhaustion. Detailed statistics and error rates are provided in Table I, anonymizing manufacturer information due to NDA policies.

TABLE I: Overview of the cross-arch. DRAM error dataset extracting from real-world ultra-scale heterogeneous clusters.

CPU arch.	DIMM mfr.	DIMMs #	DIMMs w/ CE	DIMMs w/ CE & UEs	DIMMs w/ UEs ¹	DIMMs w/ UEs in %
x86	A	530,289	6,716	466	502	7.47%
	B	337,856	4,571	560	587	12.84%
	C	355,424	7,786	1043	1,087	13.96%
	D	22,984	416	51	56	13.4%
	Total	1,223,569	19,489	2,069	2,232	11.45%
ARM	A	319,996	6,332	182	305	4.81%
	B	187,908	3,394	79	145	4.27%
	C	244,353	3,774	72	116	3.07%
	Total	752,257	13,500	333	566	4.19%
Total		1,975,826	32,989	2,402	2,798	8.45%

¹ We focus on DIMMs with (denoted as w/) CEs, excluding *Sudden UEs* (UEs that occur without any prior CE history). For each DIMM with a UE, we truncate its error log to the first UE occurrence, which is a common method used for preprocessing memory error data [25].

A. Observations

Observation 1: DIMM static properties, capacities and bit-width, have opposite UE rates in x86 and ARM architectural platforms due to distinctions in ECC. We break down the three DIMM specifications’ UE rates statistics in Figure 3. On the x86 platform, the UE rates for the three specifications are around 10%. However, on the ARM platform, the failure rate for 8-bit width is 2× to 3× higher than that of 4-bit width. This phenomenon may be attributed to the fact that ARM’s SDDC ECC implementation provides near Chipkill-level protection for x4 DIMMs. For x8 DIMMs, ARM slightly outperforms x86 by dedicating more bits to ECC verification rather than metadata. This suggests that the static properties are one of the important indicators influencing the predictive performance across x86 and ARM architectural servers.

¹ Logs from ARM servers contain address records, while x86 server logs are excluded due to distinct logging paths in AIOps.

² Bit-level data is taken from the `retry_rd_err_log_parity` register in Skylake and Cascade Lake Intel Xeon CPU but is unavailable for Huawei Kunpeng 920 processor due to the lack of these registers.

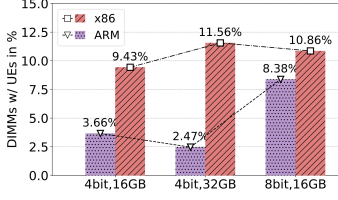


Fig. 3: Ratio of DIMMs w/ UEs in different bit widths and capacity.

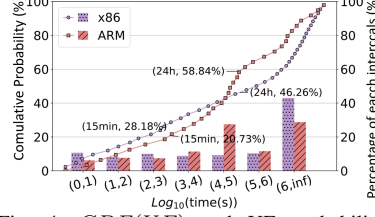


Fig. 4: $CDF(UE)$ and UE probability distribution vs. lead time.

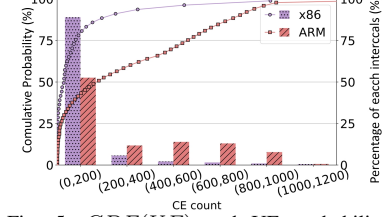


Fig. 5: $CDF(UE)$ and UE probability distribution vs. CE count.

Observation 2: The temporal distribution of CE exhibits significant discrepancies between ARM and x86 platforms.

We classify UEs as predictable or unpredictable based on the time interval between the last CE and the first UE occurrence; if this interval exceeds a threshold, the UE is deemed unpredictable. Figure 4 presents the probability distribution and Cumulative Distribution Function (CDF) of predictable UEs across x86 and ARM platforms, with the y-axis showing the proportion of predictable UEs and the x-axis representing the logarithm of prediction lead time. At shorter lead times (up to 6 hours), ARM architecture servers demonstrate slightly higher CDF probability values (2% to 8%) for predictable UEs compared to x86 servers. However, this trend reverses at the longer lead time, with x86 architecture showing a higher proportion of predictable UEs (above 12%) at the 24-hour mark. We conclude that platform differences result in varying predictability of UEs at different lead times, potentially impacting predictive performance.

Observation 3: The spatial distribution of UEs versus CE counts suggests that variations in server CPU architecture influence CE-UE progression and predictability.

Figure 5 shows the CDF and probability distribution of DIMMs with CEs leading to UEs in x86 and ARM servers. We observe that on the x86 platform, most UEs occur with few CEs, while the ARM platform shows the opposite trend, where UEs require a significant accumulation of CEs. This phenomenon may result from: varying effectiveness of CE storm suppression; or differing page offline capabilities. This correlation suggests that CE counts may be a potential predictive indicator for UE occurrences on the x86 platform. However, the predictive value of CE count for UE occurrence is substantially diminished in ARM systems.

B. Motivation

① Our observations reveal notable differences in memory failure patterns between x86 and ARM platforms, including CE and UE spatial and temporal distributions, progression characteristics, and predictability. This indicates that conventional x86-trained prediction models may underperform on ARM systems. ② Memory is crucial for system reliability, and precise predictive performance in long prediction lead time and memory failure granularity is essential in maximizing fault tolerance benefits. We propose MemSeer, a multi-grained memory failure prediction system that distinguishes memory failures in x86/ARM heterogeneous clusters.

IV. DESIGN OF MEMSEER

The overview of MemSeer is illustrated in Figure 6. It realizes multi-grained memory failure prediction in DIMM, server, and micro (page and row) levels.

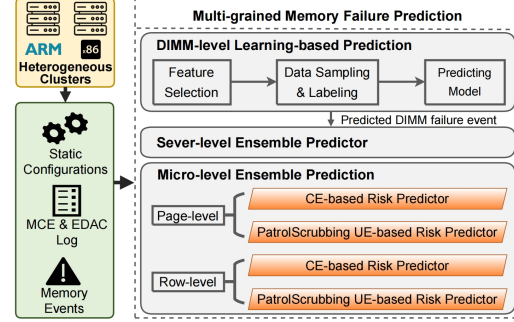


Fig. 6: The overview of MemSeer framework design. The AIOps system collects memory failure logs from various sources, such as configurations, mcelog, and memory logs, and sends them to the data lake.

A. DIMM-level Learning-based Prediction

For DIMM-level prediction, we first preprocess the data by:

- ① Ignoring memory error logs after the first UE in a DIMM, as the DIMM will be replaced and may bias model training [13].
- ② Excluding DIMMs with *Sudden UEs*.

1) *Feature Engineering*: Based on the literature review (see in §.I) and our observations, we extract four feature types for DIMM-level memory failure prediction: Static Features, Spatial Features, Temporal Features, and Bit-Level Error Features.

a) *Static Features*. These features represent the fixed DIMM characteristics of physical DRAM components.

- **CPU Model**: Different CPU models may use distinct ECC algorithms, fault tolerance technology, and MCE registers.
- **Bit Width**: The number of bits accessible per DRAM access.
- **Capacity**: Memory capacity is pertinent to DRAM density.
- **Rank Count**: The number of ranks in memory impacts the parallelism of the memory system.
- **Vendor**: Different manufacturing processes may involve distinct reliability characteristics.

b) *Spatial Features*. We design spatial features to capture the spatial distribution characteristics of CEs and UEs in each component at micro-levels.

- **Faulty_Components_Count(M_i)**: The count of components M_i with CEs, $M_i \in \{Bank, Column, Row, Cell\}$.
- **CE $\{max, avg\}$ _Count(M_i)**: The maximum and average occurrence count of CEs in each single component M_i , where $M_i \in \{Bank, Column, Row, Cell\}$.
- **Count_Error(E_i)**: The count of various errors $E_i \in \{CEs, CEs_withoutAddr^3, PatrolScrubbing_UEs\}$.

c) *Temporal Features*. The temporal distribution characteristics of CEs, including the quantity of CEs over a time period t_i and the frequency features of CE occurrences.

³CEs reported without physical addresses due to reporting limitations.

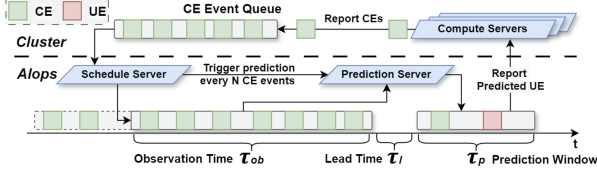


Fig. 7: The logic for DIMM-level memory failure prediction workflow.

- **Min_T_CE**(θ, t_i): The minimum time interval T for θ ⁴ CEs to appear in t_i ⁵.
- **CE_Count**(t_i): The quantity of CEs within the time t_i .
- **DIMM_Life_Span**: The time interval between the sampling time and the DIMM's operational timestamp.
- **DIMM_Error_Span**: The time interval between sampling time and first CE occurrence in a DIMM.
- **Error_Avg**: It is equal to the total number of CEs divided by the error span, reflecting the accumulation rate of CEs.

d) **Bit-Level Error Features**⁶. They characterize the distribution of error bits in DRAM DQs and Beats over a 120-hour window (t_i ⁷). These features can effectively predict UE occurrences based on memory access principles [26].

- **Distance**{*min, max*}(B_i, t_i): At the bit level, the distance between the two nearest/farthest components B_i that are affected by bit errors, where $B_i \in \{DQs, Beats\}$.
- **Bit_Level_Faulty_Count**(B_i, t_i): The number of bit-level faulty components B_i in the occurrence of a bit error, where $B_i \in \{DQs, Beats\}$.
- **Adjacent_Error**(B_i, t_i): Record the cumulative CE occurrences for each bit-level component $B_i \in \{DQs, Beats\}$.

2) *Data Sampling, Labeling and Prediction*: Observations 2 and 3 in §.III-A show fewer CE reports and a sparse distribution. We propose a CE queue event-driven sampling and prediction scheme (see Figure 7) instead of fixed time windows. A message queue in the computing cluster collects CE occurrences, which are uploaded to the Schedule Server to update the CE dataset in the observation window of τ_{ob} (6, 72, or 120 hours). When reported CEs for a DIMM reach the threshold N , the data from its observation window will be sent for UE prediction. We also consider lead time (τ_l) and prediction window (τ_p) for labeling DIMM samples as follows:

- **Positive Sample**: If the occurrence time t_{UE} of a UE falls within the sum of the time interval of the observation window and lead time, i.e., $t_{UE} \in [t_{CE_i} + \tau_l, t_{CE_i} + \tau_l + \tau_p]$.
 - **Negative Sample**: If no UE happens within $[t_{CE_i} + \tau_l, t_{CE_i} + \tau_l + \tau_p]$, this sample is labeled as a negative sample.
- In our experiments, DIMM-level prediction with $\tau_{ob} = 120$ hours and $\tau_p = 30$ days brings the best performance.

3) *Prediction Model*: DIMM-level prediction is a binary classification problem. We leverage three tree-based ML approaches, including Extreme Gradient Boosting (XGBoost) [32], Random Forest (RF) [33], and Light Gradient-Boosting Machine (LightGBM) [34]. We also investigate three neural network-based ML methods, including Self-Attention Long-Short Term Memory neural network (SA-LSTM) [35],

⁴ θ is the number of CEs and is set to 3, 5, and 7.

⁵For temporal features, t_i is set to 6 min, 6, 24, 72, and 120 hours.

⁶Available only on x86 architecture platforms (explained in §.III).

⁷For bit-level error features, we set t_i to 120 hours

Algorithm 1: Micro-level Ensemble Prediction

```

/* Rule-based Page-level Risk Predictor (CP) */
Input: The number of CEs that occur in Cell i:  $CE_{cnt}(Cell_i)$ ; Threshold of
a cell experiencing CE occurrences:  $X$ ;
Output: CE-based Risky Pages
1 for each page  $Page_j$  do
2   for each cell  $Cell_i$  in  $Page_j$  do
3     if  $CE_{cnt}(Cell_i) \geq X$  then
4       Return  $Page_j$  as a CE-based Risky Page;

/* PSUE-based Page-level risk Predictor (PP) */
Input: PatrolScrubbing UE (PSUE) count in Cell i:  $PSUE_{cnt}(Cell_i)$ ;
Output: PatrolScrubbing-based Risky Pages
5 for each page  $Page_j$  do
6   for each cell  $Cell_i$  in  $Page_j$  do
7     if  $PSUE_{cnt}(Cell_i) \neq 0$  then
8       Return  $Page_j$  as a PatrolScrubbing-based Risky Page;

/* CE-based Row-level risk Predictor (CR) */
Input: The number of cells in Row i that experience CE:
 $Cell_{CE\_cnt}(Row_i)$ ; The furthest distance between the first and the
last CEs in Row i:  $Len_{CE}(Row_i)$ ; Thresholds  $\theta_{Cell}$  and  $\theta_{Len}$ ;
Output: CE-based Risky Rows
9 for each row  $Row_i$  do
10  if  $Cell_{CE\_cnt}(Row_i) \geq \theta_{Cell}$  and  $Len_{CE}(Row_i) \geq \theta_{Len}$  then
11    Return  $Row_i$  as a CE-based Risky Row;

/* PSUE-based Row-level risk predictor (PR) */
Input: the frequency of cells that occur PatrolScrubbing UE in Row i:
 $Cell_{PSUE\_cnt}$ ; Thresholds  $\theta_{Cell}$  and  $\theta_{Len}$ ;
Output: PatrolScrubbing-based risky row
12 for each row  $Row_i$  do
13  if  $Cell_{PSUE\_cnt}(Row_i) \geq 2$  then
14    Return  $Row_i$  as a PatrolScrubbing-based Risky Row;

```

Fully-connected Neural Network (FCN) [36], and Recurrent Neural Network (RNN) [37]. These algorithms can capture complex feature correlations and are easy to implement in real-world clusters due to their lightweight footprint. We compare their predictive performance in §.V-B1 and select XGBoost as the DIMM-level predictor due to its best experimental results.

B. Server-level Ensemble Prediction

We estimate the likelihood of UEs in DIMMs within a specific lead time. Based on prior research [26, 38], we assume that UEs in server DIMMs are independent and identically distributed (i.i.d.). To derive server-level forecasts from DIMM predictions, we apply the At Least One Rule, using a logical OR on the binary failure predictions (0: Negative, 1: Positive) for all DIMMs, as shown in Equation 1. Here, $C(S_i)$ represents the predicted output for server i , and $C(D_{ij})$ indicates the prediction for DIMM j in server i , with $j \in \{1, \dots, N\}$ and N being the total DIMMs in server i . A server-level memory failure prediction triggers VM migration to avoid downtime, requiring at least 5 minutes for log transfer and migration [26, 27].

$$C(S_i) = 1 - \prod_{j=1}^N (1 - C(D_{ij})) \quad (1)$$

C. Micro-level Ensemble Prediction

Unlike DIMM-level prediction, learning methods are ineffective for micro-level predictions. Therefore, micro-level prediction considers all CEs and addressable Patrol Scrubbing UEs for identifying risks at the page and row levels, using ensemble predictions based on threshold rules. We design four predictors in Algorithm 1, detailed as follows:

1) *Rule-based Page-level Risk Identification*: Classical page-level prediction uses the $X/24$ page offlining policy [39], identifying pages with more than X faults in 24 hours for offline status. It is suggested that repeated CE occurrences in the same cell pose a high risk in correlating with UE events [27, 40]. Additionally, Patrol Scrubbing UEs notably increase the risk of subsequent UEs in the same cell. Consequently, we incorporate both CEs and Patrol Scrubbing UEs in our page-level risk identification design (Algorithm 1 lines 1 to 8).

2) *Rule-based Row-level Risk Identification*: Based on the literature [15, 16], CE can predict UE occurrences within the same column, and there's a correlation between UE and CE events in the same row. The row-level predictor in [16] uses CE frequency, CE distribution, and bit-level error features for identifying high-risk rows. Lacking bit-level logs in our ARM platform data (see in §.III), we focus on CE counts and the spatial distribution of CE and UEs at the row-level predictor (Algorithm 1 lines 9 to 14).

3) *Micro-level Risk Ensemble Prediction*: We propose page and row-level rule-based predictors, excluding others due to high costs in memory failure policies. To minimize costs, we optimize ensemble mechanisms for efficient UE prediction. The hyperparameters are calibrated to $X = 4$, $\theta_{Cell} = 5$, and $\theta_{Len} = 64$ (assuming a row length of 1024). The evaluation of micro-level risk prediction performance is discussed in §.V-B3.

V. EVALUATION

A. Experiment Setup

1) *Dataset Description*: We replay historical logs from the dataset explored in §.III to simulate an online production environment, with the dataset split by server architecture. We manipulate labels to evaluate DIMM, micro, and server-level predictions. The relationships among datasets, their utilization, and evaluation metrics are detailed in Table II.

TABLE II: Utilization of dataset.

Datasets	Bit-level logs	UE Addr	Utilization of Datasets	Evaluation Metrics
Dataset1 (x86)	✓	✗	DIMM-level prediction	R, P, F1
			Server-level prediction	R, P, F1, VMIRR
Dataset2 (ARM)	✗	✓	DIMM-level UE prediction	R, P, F1
			Micro-level prediction	Poff cost per UE
			Server-level prediction	R, P, F1, VMIRR

2) *Evaluation Metrics*: To comprehensively evaluate our proposed approach, we employ three categories of metrics:

a) Predictive Measures: We utilize standard classification metrics to assess the predictive performance:

- **Precision** = $\frac{TP}{TP+FP}$, the higher the better.
- **Recall** = $\frac{TP}{TP+FN}$, the higher the better.
- **F1-score** = $\frac{2 \times Precision \times Recall}{Precision + Recall}$, the higher the better.

TP (True Positives): correctly predicted failures. FP (False Positives): false alarms. FN (False Negatives): missed failures.

b) VM Interruption Reduction Rate (VMIRR): VMIRR is a popular metric to quantify the effectiveness of server-level memory failure prediction by live VM migrations [26, 27]. V_a is the average number of VMs per server. p_c is the proportion of cold VM migrations due to alarms and resource unavailability, and the number of VM interruptions with UE prediction is $p_c \cdot V_a(TP + FP) + V_a \cdot FN$.

$$\bullet \text{ VMIRR} = \frac{V_a(TP+FN) - (p_c \cdot V_a(TP+FP) + V_a \cdot FN)}{V_a(TP+FN)} \\ = (1 - \frac{p_c}{Precision}) \cdot Recall \text{ (the higher the better).}$$

c) Page Offlining Measures: Based on established literature [15, 16, 19, 39–41], we evaluate the effectiveness of page offlining using two key metrics:

- **Avoided UEs**: Quantifies the number of prevented UEs.
- **Page offling (Poff) per UE**: Measures the memory capacity overhead required to prevent each UE.

B. Experimental Results

We show the experimental results to comprehensively evaluate the proposed MemSeer's ① prediction performance (§.V-B1.a) and §.V-B1.b), ② feature importance (§.V-B1.c) and lead time (§.V-B1.d) sensitivity analyses, and ③ the fault tolerance value brought by multi-grained failure prediction (§.V-B2 and §.V-B3).

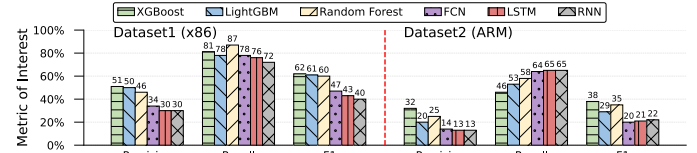


Fig. 8: Different model attempts in MemSeer for DIMM-level prediction.

1) *DIMM-level Prediction Evaluation*: **a) Comparison of various prediction models**: We compare various models discussed in §.IV-A3 for the DIMM-level predictor. Figure 8 shows the predictive performance of models on x86 and ARM architectures with a 1-second lead time and a 0.5 probability threshold. XGBoost leads with a precision of 0.51, recall of 0.81, and F1-score of 0.62 on dataset 1, while on dataset 2, it scores 0.32, 0.46, and 0.38, respectively. LightGBM achieves the highest recall of 0.87 and 0.58 for x86 and ARM datasets. Tree-based models consistently outperform NN models by 12% to 20% in precision and F1-score, though differences among models within each category are under 5%. The predictive performance of each model on x86 and ARM datasets has decreased by around 25% in F1-score, due to the absence of bit-level CE logs in ARM architecture, which has a non-negligible impact on predictive performance.

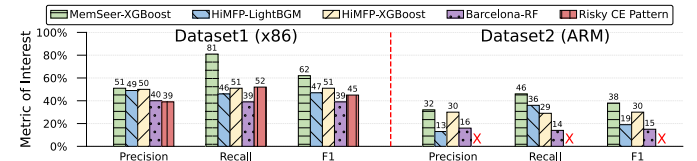


Fig. 9: Performance comparison with existing approaches.

b) Comparison with existing approaches: To evaluate DIMM-level UE prediction, we reproduce leading methods including Barcelona-Random Forest (RF) [13], HiMFP-lightGBM [26], and Risky_CE Pattern [25], fine-tuning their best models on our dataset from ultra-scale heterogeneous clusters. We compare MemSeer-XGBoost against these methods using metrics from our x86 and ARM datasets. Results in Figure 9 show XGBoost surpassing LightGBM, the best in HiMFP [26], in our heterogeneous clusters. Our method excels on both platforms, improving F1-score by 11% to 30% on dataset 1 and 8% to 20% on dataset 2. In dataset 1 (x86), our recall is at least 30% higher, while in dataset 2 (ARM), our F1-score is over 8% better, with HiMFP showing minimal difference. Risky_CE cannot be applied to ARM due to the lack of bit-level data.

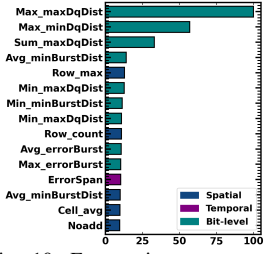


Fig. 10: Feature importance of x86 dataset

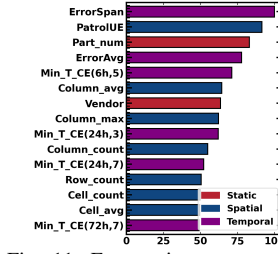


Fig. 11: Feature importance of ARM dataset

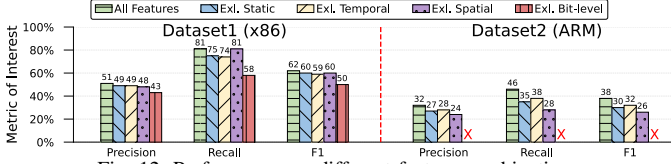


Fig. 12: Performance on different feature combinations.

c) Sensitivity I: Feature Importance Analysis and Ablation

Experiments: We conduct an ablation study to evaluate the importance of static, spatial, temporal, and bit-level features in our proposed model, MemSeer-XGBoost. Results are shown in Figure 12, and feature importance is illustrated in Figures 10 and 11 for x86 and ARM datasets, respectively. The results from the evaluation of the x86 dataset align with the findings of the previous study [26]. Bit-level features demonstrate the highest importance, with bit-level features having the highest importance, leading to decreases in precision, recall, and F1-score by 8%, 23%, and 12% when excluded. In contrast, the ARM dataset exhibits a more even distribution of importance scores among feature types, with spatial features showing noticeable effects, resulting in precision, recall, and F1-score reductions of 8%, 18%, and 8%, respectively, when excluded.

d) Sensitivity II: Lead time. In Figure 13, we assess our method’s performance in predicting DIMM-level memory failures across seven lead times based on the memory recovery strategy. We analyze page offlining within 1 minute, VM migration from 2 to 5 minutes, and an optimal 5-minute VM migration to mitigate data loss and maintain SLA during system crashes. Even with heavy workloads, a 1-hour lead time allows for manual diagnosis and DIMM replacement. Our results show that as lead time increases, all metrics decline, particularly recall, which drops by 25% and 10% for x86 and ARM, respectively. With a 6-hour lead time, we achieve 0.43 precision and 0.56 recall for x86, and 0.27 precision and 0.36 recall for ARM, resulting in F1-scores of 0.48 and 0.31. The increase in lead time hampers the model’s ability to capture temporal dependencies, reducing predictive accuracy. These findings highlight the implications of varying lead times for memory failure tolerance in real-world applications.

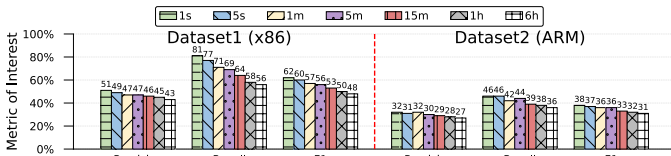


Fig. 13: Performance in different lead times.

2) Server-level Prediction and VMIRR Evaluation: In this prediction (introduced in §.IV-B), TP indicates correct predictions of servers with UEs, FP is when a normal server

TABLE III: Performance of server-level prediction and benefits of live VM migration on x86 and ARM platforms in Precision / Recall / F1-score / VMIRR.

Dataset	Lead time	MemSeer-XGBoost	HiMFP-XGBoost
Dataset1 (x86)	5min	0.41 / 0.75 / 0.53 / 40.2%	0.40 / 0.57 / 0.47 / 29.8%
	15min	0.39 / 0.70 / 0.50 / 34.7%	0.37 / 0.61 / 0.46 / 28.6%
	1h	0.36 / 0.64 / 0.46 / 28.1%	0.36 / 0.56 / 0.44 / 24.4%
Dataset2 (ARM)	5min	0.24 / 0.65 / 0.35 / 20.7%	0.24 / 0.26 / 0.25 / 8.2%
	15min	0.23 / 0.58 / 0.33 / 15.5%	0.24 / 0.24 / 0.24 / 6.8%
	1h	0.22 / 0.53 / 0.31 / 12.3%	0.23 / 0.21 / 0.22 / 5.4%

TABLE IV: Average numerical results of comparative micro-level prediction-guided page offline policies. We design four selected micro-level predictor ensembles (MemSeer-Comb1 to MemSeer-Comb4) using different combinations of CR, CP, PR and PP predictors outlined in Algorithm 1 in §.IV-C, where a ‘✓’ indicates the inclusion of a predictor and an ‘x’ indicates its exclusion.

Method	Multi-level Ensemble				Page Offlining Measures		
	CR	CP	PR	PP	Cost (pages)	Avoid UEs	Cost/UE (pages)
CR[16]	✓	x	x	x	10,452	472	23
CP[39]	x	✓	x	x	59,306	389	153
PR	x	x	✓	x	1,714	745	3
PP	x	x	x	✓	10,104	819	13
MemSeer-Comb1	x	x	✓	✓	10,317	840	13
MemSeer-Comb2	x	✓	✓	✓	68,722	962	72
MemSeer-Comb3	✓	x	✓	✓	20,329	994	21
MemSeer-Comb4	✓	✓	✓	✓	73,372	1,050	70

is incorrectly predicted to have UEs, and FN is when a UE occurs but is not predicted. We emulate VM live migration times across different loads with lead times of 5 minutes, 15 minutes, and 1 hour. We also analyze UE variations at these lead times, as shown in Figure 4. Table III presents our server-level prediction performance and VMIRR on x86 and ARM platforms. For x86, we achieve a VMIRR of 40.2% at a 5-minute lead time, with heavy VM loads reducing migrations by 28.1%. ARM platforms reach a VMIRR of 20.7% at best. Compared to HiMFP-XGBoost, MemSeer improves VMIRR by 15.2% to 34.9% on x86 and about 1.3× on ARM.

3) Micro-level Prediction and Page Offline Evaluation: We analyze micro-level page offlining benefits using Dataset 2 (ARM), as it alone has UE address logs for micro-level failure prediction. We replicate state-of-the-art OS page offlining policies [16, 39], which are also included in our approach (referred to as CR and CP). Results are shown in Table IV. CP’s cost per UE is significantly higher than the baseline, but the cost per UE for PR and PP remains lower than the baseline. Therefore, in scenarios where the priority is on the number of avoided UEs, we choose method MemSeer-Comb3, which includes three predictors outside of CP; compared to the baseline, avoided UEs increased by 110.6%, and the cost per UE is 92.4% of the original. In scenarios where precision is prioritized, we choose method MemSeer-Comb1, which improves recall by 78.0% compared to the baseline, and the cost per UE is 55.5% of the baseline.

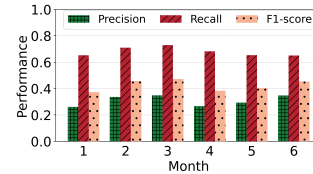


Fig. 14: OL Prediction performance.

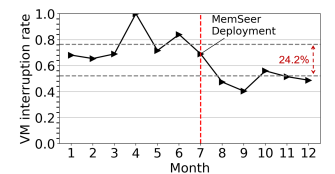


Fig. 15: OL VM interruption rate.

4) Online (OL) Results: MemSeer has been used in the online real-world heterogeneous cluster for over six months, enhancing service availability. The online precision and recall of MemSeer are shown in Figure 14. Moreover, Figure 15 illustrates the VM interruption rate before and after MemSeer’s

deployment. The Y-axis is relative due to confidentiality. The interruption rate decreased by an average of 24.2%.

VI. CONCLUSION

This paper first explores the memory failure distinctions in ultra-scale x86 and ARM heterogeneous clusters, introducing MemSeer for multi-grained memory failure prediction. It achieves 51% precision and 81% recall for x86, and 32% precision and 46% recall for ARM, reducing VM interruptions by 40.2% and 20.7%, respectively. It also significantly enhances page offlining. MemSeer online implementation in real products reduces VM interruptions by an average 24.2% in clusters.

VII. ACKNOWLEDGMENTS

We thank all the anonymous reviewers for their valuable comments. This work was supported in part by the National Key R&D Program of China (Grant No. 2023YFB4502900).

REFERENCES

- [1] J. Jiang *et al.*, "Cronus: Fault-isolated, secure and high-performance heterogeneous computing for trusted execution environment," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 124–143.
- [2] P. Patel *et al.*, "Service level agreement in cloud computing," 2009.
- [3] A. Yao *et al.*, "A memory ras system design and engineering practice in high temperature ambient data center," in *2020 19th IEEE Intersociety Conference on Thermal and Thermomechanical Phenomena in Electronic Systems (ITherm)*. IEEE, 2020, pp. 1379–1388.
- [4] A. Dutta and N. A. Touba, "Multiple bit upset tolerant memory using a selective cycle avoidance based sec-ded-daec code," in *25th IEEE VLSI Test Symposium (VTS'07)*. IEEE, 2007, pp. 349–354.
- [5] A. Note, "Intel® e7500 chipset mch intel® x4 single device data correction (x4 sddc) implementation and validation," 2002.
- [6] X. Jian *et al.*, "Low-power, low-storage-overhead chipkill correct via multi-line error correction," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2013, pp. 1–12.
- [7] Y. Luo *et al.*, "Characterizing application memory error vulnerability to optimize datacenter cost via heterogeneous-reliability memory," in *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 2014, pp. 467–478.
- [8] H. Dong *et al.*, "Memvisor: application level memory mirroring via binary translation," in *2012 IEEE International Conference on Cluster Computing*. IEEE, 2012, pp. 562–565.
- [9] X. Guo *et al.*, "Sanitizer: Mitigating the impact of expensive ecc checks on stt-mram based main memories," *IEEE Transactions on Computers*, vol. 67, no. 6, pp. 847–860, 2017.
- [10] S. Scargall and S. Scargall, "Reliability, availability, and serviceability (ras)," *Programming Persistent Memory: A Comprehensive Guide for Developers*, pp. 333–346, 2020.
- [11] I. Giurgiu *et al.*, "Predicting dram reliability in the field with machine learning," in *Proceedings of the 18th ACM/IFIP/USENIX Middleware Conference: Industrial Track*, 2017, pp. 15–21.
- [12] P. Nikolaou *et al.*, "Modeling the implications of dram failures and protection techniques on datacenter tco," in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015, pp. 572–584.
- [13] I. Boixaderas *et al.*, "Cost-aware prediction of uncorrected dram errors in the field," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–15.
- [14] X. Du and C. Li, "Predicting uncorrectable memory errors from the correctable error history: No free predictors in the field," in *The International Symposium on Memory Systems*, 2021, pp. 1–10.
- [15] X. Du and C. Li, "Combining error statistics with failure prediction in memory page offlining," in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 127–132.
- [16] X. Du *et al.*, "Fault-aware prediction-guided page offlining for uncorrectable memory error prevention," in *2021 IEEE 39th International Conference on Computer Design (ICCD)*. IEEE, 2021, pp. 456–463.
- [17] F. Zhang *et al.*, "A survey on virtual machine migration: Challenges, techniques, and open issues," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 2, pp. 1206–1243, 2018.
- [18] X. Du and C. Li, "Memory failure prediction using online learning," in *Proceedings of the International Symposium on Memory Systems*, 2018, pp. 38–49.
- [19] X. Du *et al.*, "Predicting uncorrectable memory errors for proactive replacement: An empirical study on large-scale field data," in *2020 16th European Dependable Computing Conference (EDCC)*. IEEE, 2020, pp. 41–46.
- [20] F. Yu *et al.*, "Dram failure prediction in large-scale data centers," in *2021 IEEE International Conference on Joint Cloud Computing (JCC)*. IEEE, 2021, pp. 1–8.
- [21] Z. Cheng *et al.*, "An in-depth correlative study between dram errors and server failures in production data centers," in *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2022, pp. 262–272.
- [22] J. Bogatinovski *et al.*, "First ce matters: On the importance of long term properties on memory failure prediction," in *2022 IEEE International Conference on Big Data (Big Data)*. IEEE, 2022, pp. 4733–4736.
- [23] L. Mukhanov *et al.*, "Workload-aware dram error prediction using machine learning," in *2019 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2019, pp. 106–118.
- [24] X. Wang *et al.*, "On workload-aware dram failure prediction in large-scale data centers," in *2021 IEEE 39th VLSI Test Symposium (VTS)*. IEEE, 2021, pp. 1–6.
- [25] C. Li *et al.*, "From correctable memory errors to uncorrectable memory errors: what error bits tell," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 01–14.
- [26] Q. Yu *et al.*, "Himfp: Hierarchical intelligent memory failure prediction for cloud service reliability," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2023, pp. 216–228.
- [27] P. Zhang *et al.*, "Predicting dram-caused node unavailability in hyper-scale clouds," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2022, pp. 275–286.
- [28] Q. Yu *et al.*, "Investigating memory failure prediction across cpu architectures," 2024. [Online]. Available: <https://arxiv.org/abs/2406.05354>
- [29] M. Hillenbrand *et al.*, "Multiple physical mappings: Dynamic dram channel sharing and partitioning," in *Proceedings of the 8th Asia-Pacific Workshop on Systems*, 2017, pp. 1–9.
- [30] D. Technologies, "Memory errors and dell emc poweredge yx4x server memory ras features," Tech. Rep., 2020. [Online]. Available: https://dl.dell.com/manuals/common/dellemc_poweredge_yx4x_memoryras_v1_1.pdf
- [31] Y. Han *et al.*, "Data-aware dram refresh to squeeze the margin of retention time in hybrid memory cube," in *2014 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 2014, pp. 295–300.
- [32] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.
- [33] S. J. Rigatti, "Random forest," *Journal of Insurance Medicine*, vol. 47, no. 1, pp. 31–39, 2017.
- [34] G. Ke *et al.*, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, 2017.
- [35] W. Li *et al.*, "Bidirectional lstm with self-attention mechanism and multi-channel features for sentiment classification," *Neurocomputing*, vol. 387, pp. 63–77, 2020.
- [36] T. N. Sainath *et al.*, "Convolutional, long short-term memory, fully connected deep neural networks," in *2015 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2015, pp. 4580–4584.
- [37] L. R. Medsker and L. Jain, "Recurrent neural networks," *Design and Applications*, vol. 5, no. 64-67, p. 2, 2001.
- [38] B. Schroeder *et al.*, "Dram errors in the wild: a large-scale field study," *ACM SIGMETRICS Performance Evaluation Review*, vol. 37, no. 1, pp. 193–204, 2009.
- [39] C. H. Costa *et al.*, "A system software approach to proactive memory-error avoidance," in *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2014, pp. 707–718.
- [40] A. A. Hwang *et al.*, "Cosmic rays don't strike twice: Understanding the nature of dram errors and the implications for system design," *ACM SIGPLAN Notices*, vol. 47, no. 4, pp. 111–122, 2012.
- [41] J. Meza *et al.*, "Revisiting memory errors in large-scale production data centers: Analysis and modeling of new trends from the field," in *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 2015, pp. 415–426.